

1. ¿Qué es la persistencia?

Es la capacidad de **guardar datos de forma permanente**

Ejemplos:

- Guardar usuarios
- Guardar pedidos
- Guardar configuraciones

Hasta ahora:

- Guardábamos texto → "Juan, 25"
 - Ahora guardamos **objetos completos**
-

♦ 2. Modos de acceso a ficheros

✓ Acceso secuencial

- Se lee de principio a fin
- Como una lista

Ejemplo: leer un `.txt`

✓ Acceso directo (aleatorio)

- Puedes ir a una posición concreta
- Como un array

👉 Ejemplo: `RandomAccessFile` (más avanzado)

♦ 3. Ficheros binarios

No son legibles por humanos

Guardan datos en formato interno (más eficiente)

💡 Aquí entra la serialización

4. ¿Qué es la serialización?

Convertir un objeto en bytes para:

- guardarlo en archivo
- enviarlo por red



Lo contrario:

Deserialización → recuperar el objeto

5. Requisito clave: Serializable

Para poder serializar una clase:

```
import java.io.Serializable;
```

```
public class Persona implements Serializable {  
    String nombre;  
    int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

- ✓ No hace falta implementar métodos
 - ✓ Es una interfaz "marcadora"
-

6. Guardar objetos (ObjectOutputStream)

Ejemplo

```
import java.io.FileOutputStream;  
import java.io.ObjectOutputStream;  
import java.io.IOException;  
  
public class GuardarObjeto {  
    public static void main(String[] args) {  
        Persona p = new Persona("Gabriel", 30);  
  
        try (ObjectOutputStream oos =  
            new ObjectOutputStream(new FileOutputStream("persona.dat"))) {
```

```
        oos.writeObject(p);

        System.out.println("Objeto guardado");

    } catch (IOException e) {
        System.out.println("Error");
    }
}
}
```

7. Leer objetos (ObjectInputStream)

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

public class LeerObjeto {
    public static void main(String[] args) {
        try (ObjectInputStream ois =
            new ObjectInputStream(new FileInputStream("persona.dat"))) {

            Persona p = (Persona) ois.readObject();

            System.out.println(p.nombre + " - " + p.edad);

        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error");
        }
    }
}
```

Explicación importante

- `writeObject()` → guarda el objeto
 - `readObject()` → lo recupera
 - Hay que hacer **casting**
-

8. Serializar varios objetos

```
oos.writeObject(new Persona("Ana", 20));
oos.writeObject(new Persona("Luis", 25));
```

Leer varios objetos

```
try {
    while (true) {
        Persona p = (Persona) ois.readObject();
        System.out.println(p.nombre);
    }
} catch (EOFException e) {
    System.out.println("Fin del archivo");
}
```

💡 Se usa `EOFException` para detectar final

9. Atributos que NO se serializan

👉 Usamos `transient`

```
transient String password;
```

- ✓ No se guarda en el archivo
 - ✓ Útil para datos sensibles
-

10. ID de versión (importante en teoría)

```
private static final long serialVersionUID = 1L;
```

👉 Sirve para controlar versiones de la clase

11. Ejemplo completo (MUY importante)

Guardar lista de objetos

```
import java.io.*;
import java.util.ArrayList;

public class GuardarLista {
    public static void main(String[] args) {
        ArrayList<Persona> lista = new ArrayList<>();

        lista.add(new Persona("Ana", 20));
        lista.add(new Persona("Luis", 25));
    }
}
```

```

try (ObjectOutputStream oos =
    new ObjectOutputStream(new FileOutputStream("personas.dat"))) {

    oos.writeObject(lista);

} catch (IOException e) {
    System.out.println("Error");
}
}
}

```

Leer lista

```

import java.io.*;
import java.util.ArrayList;

public class LeerLista {
    public static void main(String[] args) {
        try (ObjectInputStream ois =
            new ObjectInputStream(new FileInputStream("personas.dat"))) {

            ArrayList<Persona> lista =
                (ArrayList<Persona>) ois.readObject();

            for (Persona p : lista) {
                System.out.println(p.nombre);
            }

        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error");
        }
    }
}

```

12. Problemas típicos

- ✗ No implementar `Serializable` → error
 - ✗ Cambiar la clase después de guardar → error
 - ✗ No hacer casting → error
-

Resumen

- Serializar = guardar objetos como bytes

- `Serializable` → obligatorio
- `ObjectOutputStream` → escribir
- `ObjectInputStream` → leer
- `transient` → no guardar atributo
- Se pueden guardar listas completas

EJERCICIOS

1. Guardar objeto Persona

Serializar un objeto simple.

2. Leer objeto Persona

Deserializar y mostrar datos.

3. Guardar varios objetos

Escribir 3 personas en archivo.

4. Leer varios objetos

Leer hasta EOFException.

5. Lista serializada

Guardar ArrayList de personas.

6. Leer lista completa

Recuperar ArrayList desde archivo.

7. Atributo transient

Crear contraseña que no se guarde.

8. Modificar clase y probar error

Cambiar clase y ver problema de serialVersionUID.

9. Guardar productos

Clase Producto con nombre y precio.

10. Producto más caro

Leer lista y mostrar el más caro.